

Content Negotiation

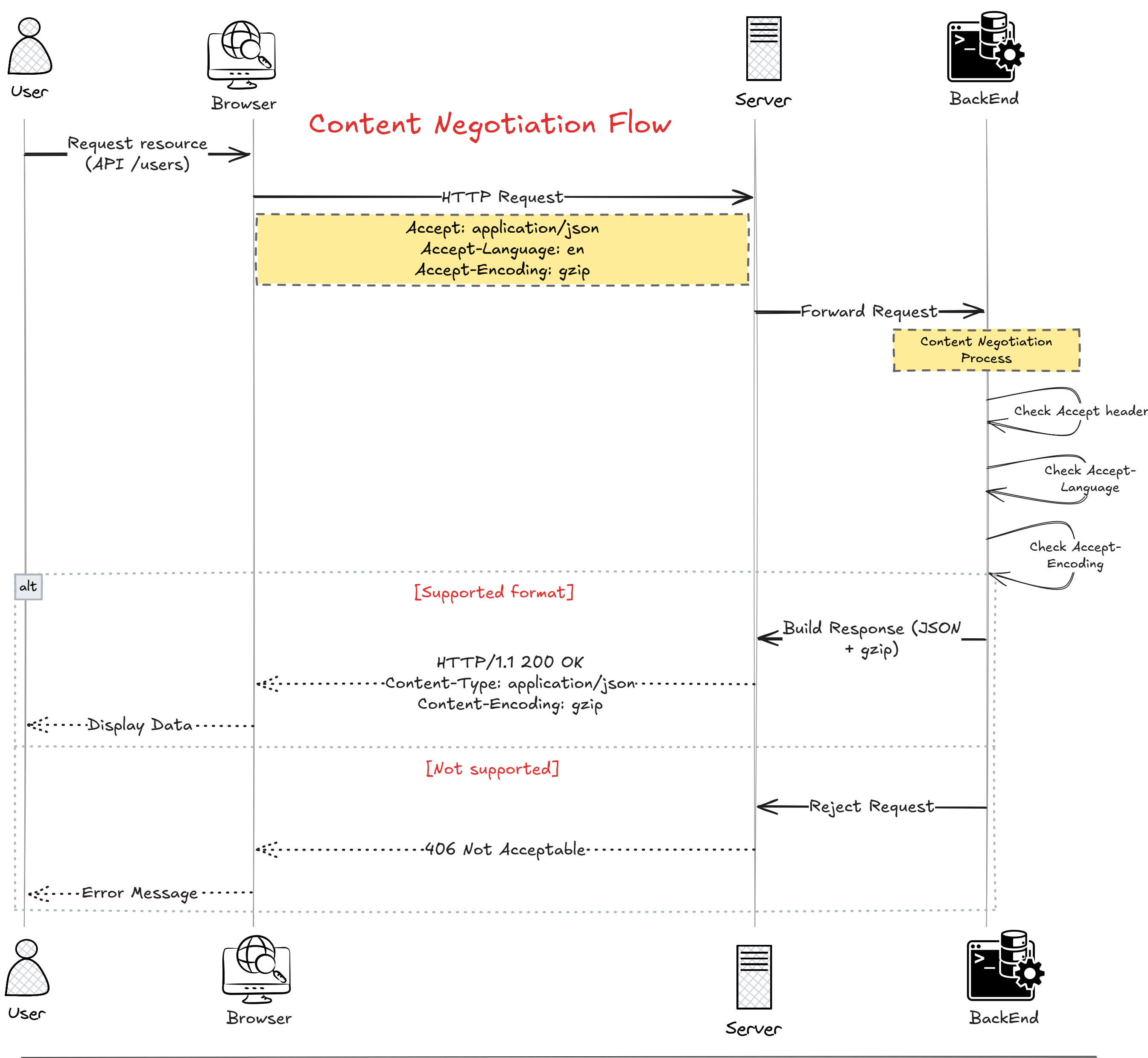
Content Negotiation is a mechanism where the client (browser or API) and server agree on the best representation of a resource.

Request

```
GET /data HTTP/1.1
Host: example.com
Accept: application/json
Accept-Language: en-US
Accept-Encoding: gzip
```

Server Response:

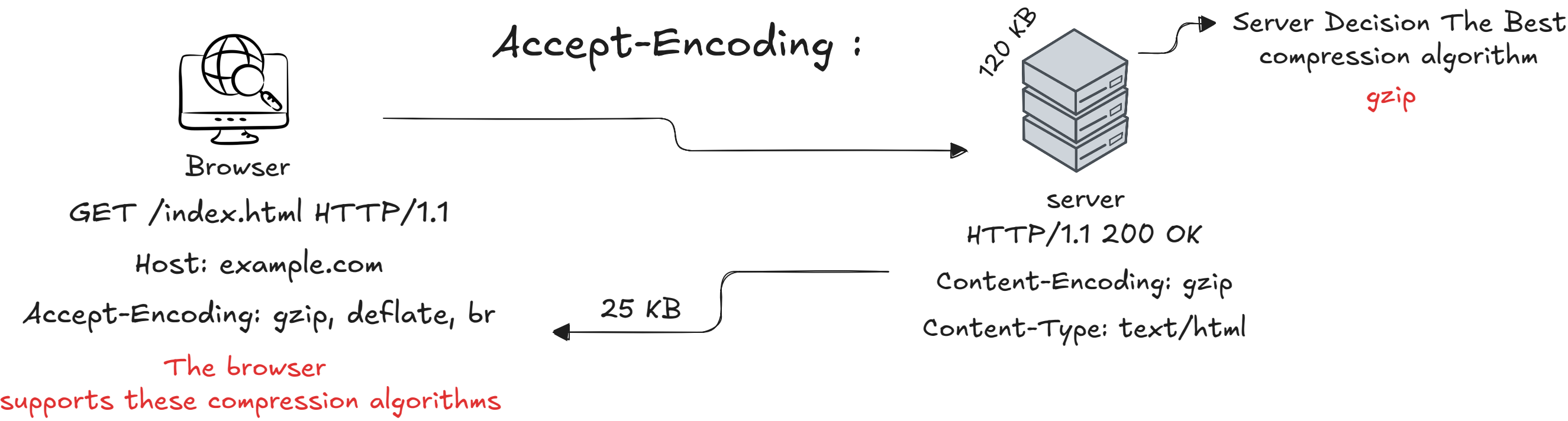
```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Language: en-US
Content-Encoding: gzip
{ "name": "Mahmoud", "role": "Student" }
```



HTTP-Based Compression

HTTP compression is the process where: the server compresses the response before sending it to the client.

How they Agree of the Compression ?



Persistent Connection & Keep-Alive

Originally in HTTP/1.0; each request opened a new TCP connection and closed it after the response. A Persistent Connection allows: multiple HTTP requests and responses to use the same TCP connection. This reduces overhead and improves performance.

Keep-Alive Header
Connection: keep-alive

Note that in HTTP/1.1 Persistent connections is a default behavior. How to close that connection? use Connection: close

Handling large Requests and Responses

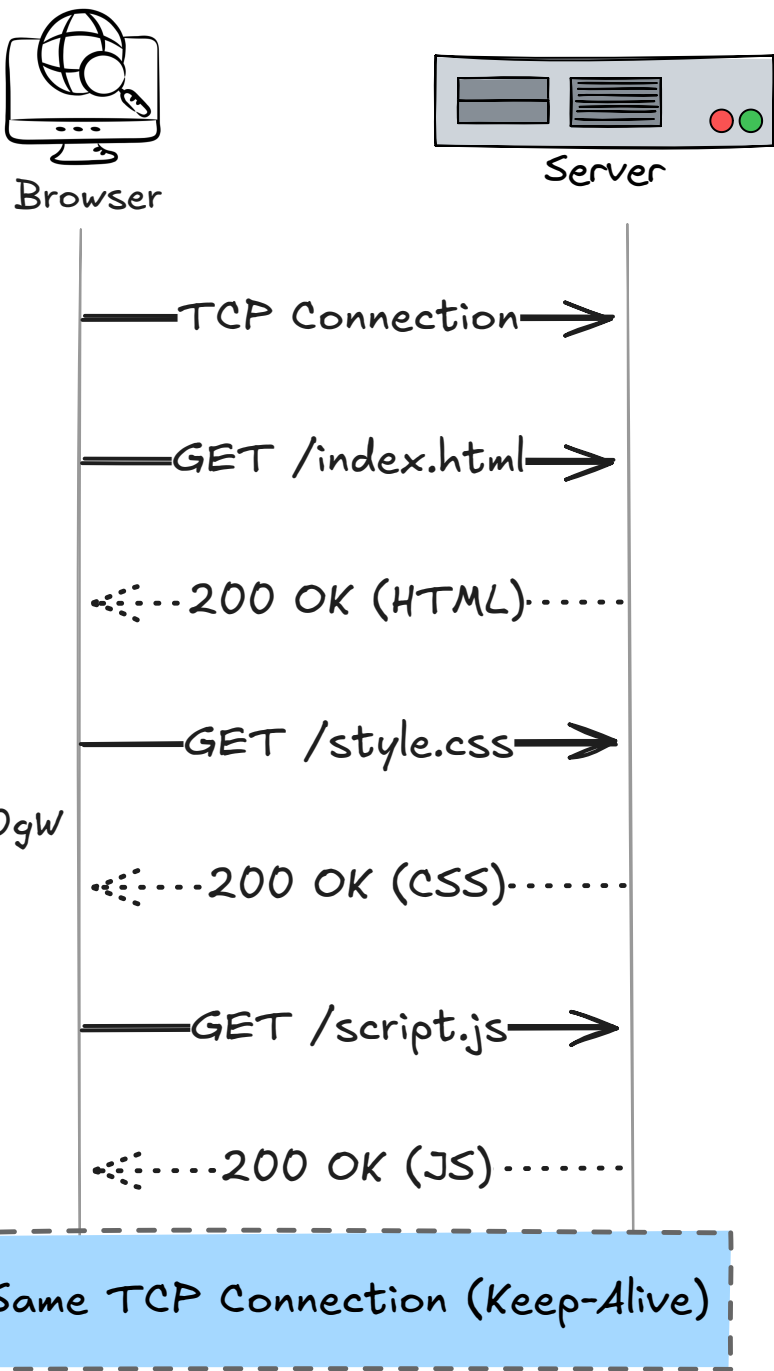
1. Multipart Boundaries

POST /upload HTTP/1.1

Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="username"

Mahmoud
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="profile_pic"; filename="avatar.jpg"
Content-Type: image/jpeg
-----WebKitFormBoundary7MA4YWxkTrZu0gW--

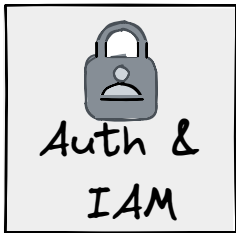


HTTPS

HyperText Transfer Protocol Secure
HTTP + SSL/TLS → HTTPS

Before data is sent, a Handshake occurs between browser and server to establish encryption. All data is encrypted and protected from eavesdropping or tampering (Integrity & Confidentiality).

Data encryption (Encryption)
Server authentication (Authentication)
Data integrity (Data Integrity)



Auth & IAM

SSL

Secure Sockets Layer

Encrypts data between client and server and verifies server identity.

Old SSL versions have security vulnerabilities (e.g., POODLE, BEAST attacks). Modern sites don't use old SSL, they use TLS.

TLS

Transport Layer Security

Stronger encryption, better performance, protects against MitM attacks.

TLS Handshake



Private



Public

TLS 1.0 → Old, insecure
TLS 1.2 → Very common, secure
TLS 1.3 → Latest, faster, more secure, fewer handshake steps